



Special Course in Software Engineering '26

Project – Testing

Important

- Work in groups
 - Therefore, you must sit with your group members
 - Communicating over WeChat or similar application is not acceptable
 - If you don't like this, then you are free to leave, and you'll be marked absent

Refine your developer agent

- Your developer_agent may have produced some navigation logic. It's probably correct.
- However, not every group's navigation logic may look the same, which is also fine.
- But now you must make sure the developer agent produces a main function called **decide_next_move(state)**
 - Where '**state**' will be the input the robot gets from its sensor. Don't worry about the 'sensor' part.
- Remember, the original navigation logic should be retained. It's only mandatory that other agents/programs call only the **decide_next_move(state)** and no other helper functions inside it

Testing strategy

You must use two levels of testing strategies

- **Level 1 – Agent / Pipeline *smoke* tests**
 - Run the actual agents with their real inputs
 - Let each agent's own validation accept or reject its output
- **Level 2 – Behavioral testing**
 - Call `decide_next_move(state)` with known states and check the returned action.

Test the analyst agent

Create `test_analyst.py`.

- Take the original `brief.txt` file, send it to the analyst agent, and let the agent run as usual.
- The output from the agent must be stored in the artifacts folder you created earlier. It must be stored under `requirements.json` and displayed after the test ends
- Ensure the agent turns the brief text into a usable requirements artifact

Test the planner agent

Create `test_planner.py`.

- Take the artifact `requirements.json` file, send it to the planner agent, and let the agent run as usual.
- The output from the agent must be stored in the artifacts folder under `plan.json` and displayed after the test ends
- Ensure the agent turns the requirements artifact into a plan that developer agent can use

Test the developer agent

Create `test_developer.py`.

- Take the artifact `plan.json` file, send it to the developer agent, and let the agent run it as usual.
- The output from the agent must be stored in the artifacts folder under `navigation_logic.py` and displayed after the test ends
- The different logic in `navigation_logic.py` you may have is fine, but you must make sure that the `decide_next_move(state)`, that other functions/programs will call, is created

Test the navigation logic

Create `test_generated_navigation_logic.py`

- Use the `decide_next_move` from the navigation logic artifact
- Create several test cases to test every known robot state. A sample test case is given in the `test_generated_navigation_logic.py` shared with you
 - Pass the "state" from each case to the developer agent, let it run as usual, and check the result is a valid action and that it is also the correct expected move
- Try to test even those states where the generated logic may not work as expected

Introduce a bug

Now, you'll add a bug intentionally into your `navigation_logic.py` to see if the test cases actually catch it

- Change something important in your `navigation_logic.py` like making one test case return an incorrect direction
- Run the tests again. At least one test should fail.
- If the bug is not detected, then add a test to detect that bug. Test it again to confirm the test detects it.
- Then remove the bug from `navigation_logic.py` and run all the test cases again
- Test cases are useful only if it detects bugs and other issues, not just because it prints '**passed**'.

If a test fails...

Do not immediately ask AI to rewrite tests if a test fails even after you removed the bug, or even before adding it

- Find out where the problem is:
 - Is the `navigation_logic.py` wrong?
 - Does your developer agent doesn't understand the plan?
 - Or is the `plan.json` wrong?
 - Or is there an issue during the requirements stage?

Finally, you should have...

- `test_analyst.py`
- `test_planner.py`
- `test_developer.py`
- `test_generated_navigation_logic.py`
- `artifacts/requirements.json`
- `artifacts/plan.json`
- `generated/navigation_logic.py`

That's all for this practice